



Shri Vile Parle Kelavani Mandal's

DWARKADAS J.SANGHVI COLLEGE OF ENGINEERING

(Empowered Autonomous College Affiliated to the University of Mumbai)



Department of Information Technology

NAME: RONAK KHATRI

ROLL NO: I145

SAP ID: 60003250067

BATCH: I3-1

DATE: 02/10/2026

S.Y. BTech (IT)

SUB: Database Management System

Self-Learning Topic 1

To implement Procedures, Functions and Cursors

AIM: To study and implement procedure, function and cursors

Course Outcome: Construct SQL queries to perform operations on the database.

THEORY:

A procedure is a PL/SQL block which performs one or more specific tasks. It is just like procedures in other programming languages.

The procedure contains a header and a body.

Header: The header contains the name of the procedure and the parameters or variables passed to the procedure.

Body: The body contains a declaration section, execution section and exception section similar to a general PL/SQL block.

Syntax for creating procedure:

```
CREATE [OR REPLACE] PROCEDURE procedure_name  
[(parameter1 [mode1] datatype1, parameter2 [mode2] datatype2)]  
AS BEGIN  
    <Body of the code>  
END;
```

Example:



Shri Vile Parle Kelavani Mandal's

DWARKADAS J.SANGHVI COLLEGE OF ENGINEERING

(Empowered Autonomous College Affiliated to the University of Mumbai)



Department of Information Technology

In this example, we are going to insert record in user table. So you need to create user table first.

Table creation:

```
create table user(id number(10) primary key, name varchar2(100));
```

Now write the procedure code to insert record in user table.

Procedure Code:

```
create or replace procedure INSERTUSER (id IN NUMBER, name IN VARCHAR2)
```

```
is
```

```
begin insert into user
```

```
values(id,name); end;
```

```
/
```

PL/SQL program to call procedure

Let's see the code to call above created procedure.

```
BEGIN insertuser(101,'Rahul');
```

```
dbms_output.put_line('record inserted successfully');
```

```
END;
```

```
/
```

Now, see the "USER" table, you will see one record is inserted.

ID	NAME
101	Rahul



Shri Vile Parle Kelavani Mandal's

DWARKADAS J.SANGHVI COLLEGE OF ENGINEERING

(Empowered Autonomous College Affiliated to the University of Mumbai)



Department of Information Technology

PL/SQL Drop Procedure

Syntax for drop procedure

DROP PROCEDURE procedure_name;

Example of drop procedure

DROP PROCEDURE pro1;

FUNCTIONS:

The PL/SQL Function is very similar to PL/SQL Procedure. The main difference between procedure and a function is, a function must always return a value, and on the other hand a procedure may or may not return a value. Except this, all the other things of PL/SQL procedure are true for PL/SQL function too.

Syntax to create a function:

CREATE [OR **REPLACE**] **FUNCTION** function_name [parameters]

[(parameter_name [IN | **OUT** | IN **OUT**] type [, ...])]

RETURN return_datatype

{**IS** | **AS**} **BEGIN**

< function_body >

END [function_name];

Let's see a simple example to **create a function**.

CREATE or replace FUNCTION FetchAccountBal (varAcno IN Number) RETURN

NUMBER IS

varCurrentBal Number (12,2);

BEGIN

Select Bal INTO varCurrentBal FROM ACCOUNTS

Where Acno=varAcno;



Shri Vile Parle Kelavani Mandal's

DWARKADAS J.SANGHVI COLLEGE OF ENGINEERING

(Empowered Autonomous College Affiliated to the University of Mumbai)



Department of Information Technology

```
RETURN (varCurrentBal);
```

```
END;
```

To execute a function:

```
Select FetchAccountBal(1) from dual;
```

CURSORS

When an SQL statement is processed, Oracle creates a memory area known as context area. A cursor is a pointer to this context area. It contains all information needed for processing the statement. In PL/SQL, the context area is controlled by Cursor. A cursor contains information on a select statement and the rows of data accessed by it.

A cursor is used to referred to a program to fetch and process the rows returned by the SQL statement, one at a time. There are two types of cursors:

- Implicit Cursors: It is automatically created by oracle while executing an SQL query ○

Explicit Cursors: cursor opened for processing data through a PL/SQL block (user defined)

CURSOR ATTRIBUTES

- **%ISOPEN**
 - It is a Boolean variable which evaluates to TRUE if the cursor is opened , otherwise to false
 - Always returns **FALSE** for **implicit cursors**, because Oracle closes the SQL cursor automatically after executing its associated SQL statement.
- **%FOUND**
 - Returns TRUE if an INSERT, UPDATE, or DELETE statement affected one or more rows or a SELECT INTO statement returned one or more rows.



Department of Information Technology

- Otherwise, it returns FALSE.
- **%NOTFOUND**
- It returns TRUE if an INSERT, UPDATE, or DELETE statement affected no rows, or a SELECT INTO statement returned no rows.
- Otherwise, it returns FALSE.
- **%ROWCOUNT**
- Returns the number of rows affected by an INSERT, UPDATE, or DELETE statement, or returned by a SELECT INTO statement.

Example of Implicit Cursor DECLARE

```
total_rows number(2);  
  
BEGIN  
  
    UPDATE customers  
  
    SET salary = salary + 500;    IF sql%notfound THEN  
  
dbms_output.put_line('no customers selected');    ELSIF  
  
sql%found THEN        total_rows := sql%rowcount;  
  
dbms_output.put_line( total_rows || ' customers selected ');  
  
    END IF;  
  
END;
```

Example of Explicit Cursor:

```
DECLARE  
  
    emp_rec emp_tbl%rowtype;  
  
    CURSOR emp_cur IS
```



Department of Information Technology

```
SELECT *  
FROM emp_tbl  
WHERE salary > 10;  
  
BEGIN  
    OPEN emp_cur;  
    FETCH emp_cur INTO emp_rec;    dbms_output.put_line  
(emp_rec.first_name || ' ' || emp_rec.last_name);  
    CLOSE emp_cur;  
END;
```

Implementation in SQL

Students are suggested to use Oracle or MySQL for the implementation.

1. Create table order_master with the following columns.

Column name	Data type	Size
Order_no	Number	
Order_date	Date	
Order_status	Char	1
Delete_date	Date	

```
CREATE TABLE order_master (  
    Order_no NUMBER PRIMARY KEY,  
    Order_date DATE,  
    Order_status CHAR(1),
```



Department of Information Technology

Delete_date DATE

);

Table created.

0.25 seconds

2. Insert values into the order_master table.

```
INSERT INTO order_master VALUES (101, 'JAN-01-26', 'C', NULL);
```

```
INSERT INTO order_master VALUES (102, 'JAN-09-26', 'N', NULL);
```

```
INSERT INTO order_master VALUES (103, 'JAN-12-26', 'C', NULL);
```

```
INSERT INTO order_master VALUES (104, 'JAN-16-26', 'N', NULL);
```

```
INSERT INTO order_master VALUES (105, 'JAN-20-26', 'C', NULL);
```

```
SELECT * FROM order_master;
```

1 row(s) inserted.

0.01 seconds

ORDER_NO	ORDER_DATE	ORDER_STATUS	DELETE_DATE
101	01/01/0026	C	-
102	01/09/0026	N	-
103	01/12/0026	C	-
104	01/16/0026	N	-
105	01/20/0026	C	-

5 rows returned in 0.02 seconds

[Download](#)



Department of Information Technology

3. Create a procedure and a function separately that accepts order_no as the input and returns the order_status.

PROCEDURE

```
CREATE OR REPLACE PROCEDURE get_order_status_proc (  
    p_order_no IN NUMBER,  
    p_status OUT CHAR  
) AS  
BEGIN  
    SELECT Order_status  
    INTO p_status  
    FROM Order_master  
    WHERE Order_no = p_order_no;  
EXCEPTION  
    WHEN NO_DATA_FOUND THEN  
        p_status := NULL;  
        DBMS_OUTPUT.PUT_LINE('Order number ' || p_order_no || ' not found.');
```

END;
/

Procedure created.

0.15 seconds



Shri Vile Parle Kelavani Mandal's

DWARKADAS J.SANGHVI COLLEGE OF ENGINEERING

(Empowered Autonomous College Affiliated to the University of Mumbai)



Department of Information Technology

FUNCTION

```
CREATE OR REPLACE FUNCTION get_order_status_func (
```

```
    p_order_no IN NUMBER
```

```
) RETURN CHAR AS
```

```
    v_status CHAR(1);
```

```
BEGIN
```

```
    SELECT Order_status
```

```
    INTO v_status
```

```
    FROM order_master
```

```
    WHERE Order_no = p_order_no;
```

```
    RETURN v_status;
```

```
EXCEPTION
```

```
    WHEN NO_DATA_FOUND THEN
```

```
        RETURN NULL;
```

```
END;
```

```
/
```

```
Function created.
```

```
0.03 seconds
```



Department of Information Technology

CALL

DECLARE

```
v_status CHAR(1);
```

BEGIN

```
get_order_status_proc(101, v_status);
```

```
DBMS_OUTPUT.PUT_LINE('Order Status: ' || v_status);
```

END;

/

```
SELECT get_order_status_func(101) AS order_status FROM DUAL;
```

```
Order Status: C
```

```
Statement processed.
```

```
0.00 seconds
```

ORDER_STATUS
C

1 rows returned in 0.00 seconds

[Download](#)

4. Implement an implicit and explicit cursor to retrieve all the details of the orders whose status is 'C' where C indicates order is complete and N indicates incomplete.



Department of Information Technology

IMPLICIT

BEGIN

UPDATE order_master

SET delete_date = SYSDATE

WHERE order_status = 'C';

DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' orders found and updated with status C');

END;

/

3 orders found and updated with status C

1 row(s) updated.

0.01 seconds

DISPLAY

SELECT *

FROM order_master

WHERE order_status = 'C';

ORDER_NO	ORDER_DATE	ORDER_STATUS	DELETE_DATE
101	01/01/0026	C	10/02/2026
103	01/12/0026	C	10/02/2026
105	01/20/0026	C	10/02/2026

3 rows returned in 0.00 seconds

[Download](#)



Shri Vile Parle Kelavani Mandal's

DWARKADAS J.SANGHVI COLLEGE OF ENGINEERING

(Empowered Autonomous College Affiliated to the University of Mumbai)



Department of Information Technology

EXPLICIT CURSOR

DECLARE

CURSOR order_cur IS

SELECT order_no, order_date, order_status, delete_date

FROM order_master

WHERE order_status = 'C';

order_rec order_cur%ROWTYPE;

BEGIN

OPEN order_cur;

LOOP

FETCH order_cur INTO order_rec;

EXIT WHEN order_cur%NOTFOUND;

DBMS_OUTPUT.PUT_LINE('Order No: ' || order_rec.order_no || ' | Date: '
|| order_rec.order_date || ' | Status: ' || order_rec.order_status || ' | Del Date: ' ||
order_rec.delete_date);

END LOOP;

CLOSE order_cur;

END;

/

Order No: 101 | Date: 01/01/0026 | Status: C | Del Date: 10/02/2026

Order No: 103 | Date: 01/12/0026 | Status: C | Del Date: 10/02/2026

Order No: 105 | Date: 01/20/0026 | Status: C | Del Date: 10/02/2026

Statement processed.

0.01 seconds